

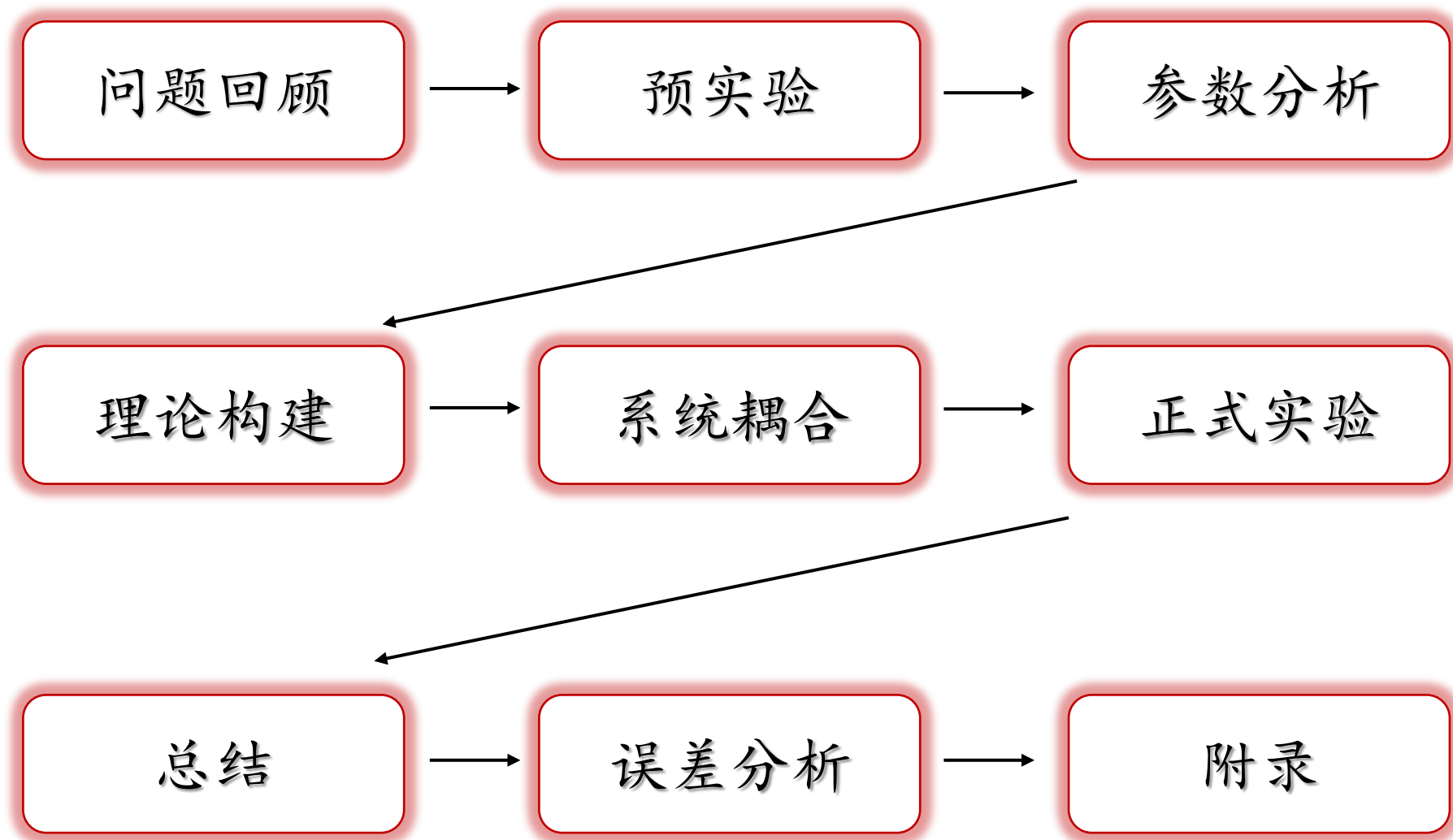


四川大學
SICHUAN UNIVERSITY

尺子把戏

Ruler Trick

匹兹堡学院 何岱洋

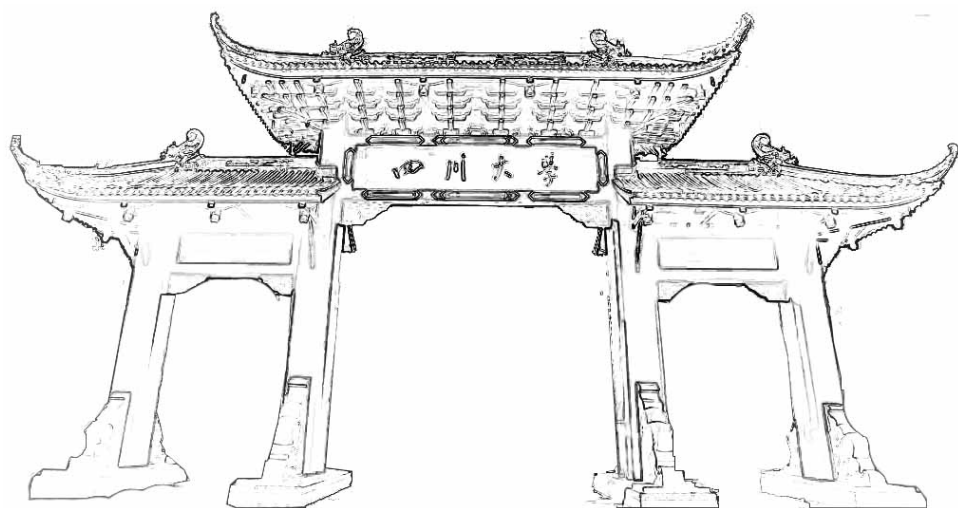




- Place a ruler on the edge of a table, and throw a ball at its **free end**. The ruler will fall. However, if you **cover a part of the ruler with a piece of paper** and **repeat the throw**, then the ruler will **remain on the table** while **the ball will bounce off it**. **Explain** this phenomenon, and investigate the **relevant parameters**.
- 将尺子放在桌子的边缘，并朝其**自由端**扔球。尺子会掉下来。但是，如果你**用纸张覆盖尺子的一部分**并**重复投掷**，则尺子将**保持在桌子上**，而**球将从上面弹起**。**解释**这一现象，并调查**相关参数**。



- 1、研究内容（contents）：“尺子把戏”现象的成因及相关参数
（Causes of the “Ruler Trick” Phenomenon and Relevant Parameters）
- 2、研究对象：尺子、纸张、小球、空气
（Ruler, Paper, Ball, Air）
- 3、问题关键：
 - ①方式：向自由端扔球（throw a ball at its free end）
 - ②现象：尺子保持在桌子上，球弹起（the ruler will remain on the table while the ball will bounce off it）
 - ③要求：解释现象、调查参数（Explain this phenomenon, and investigate the relevant parameters）



预实验

第一次预实验



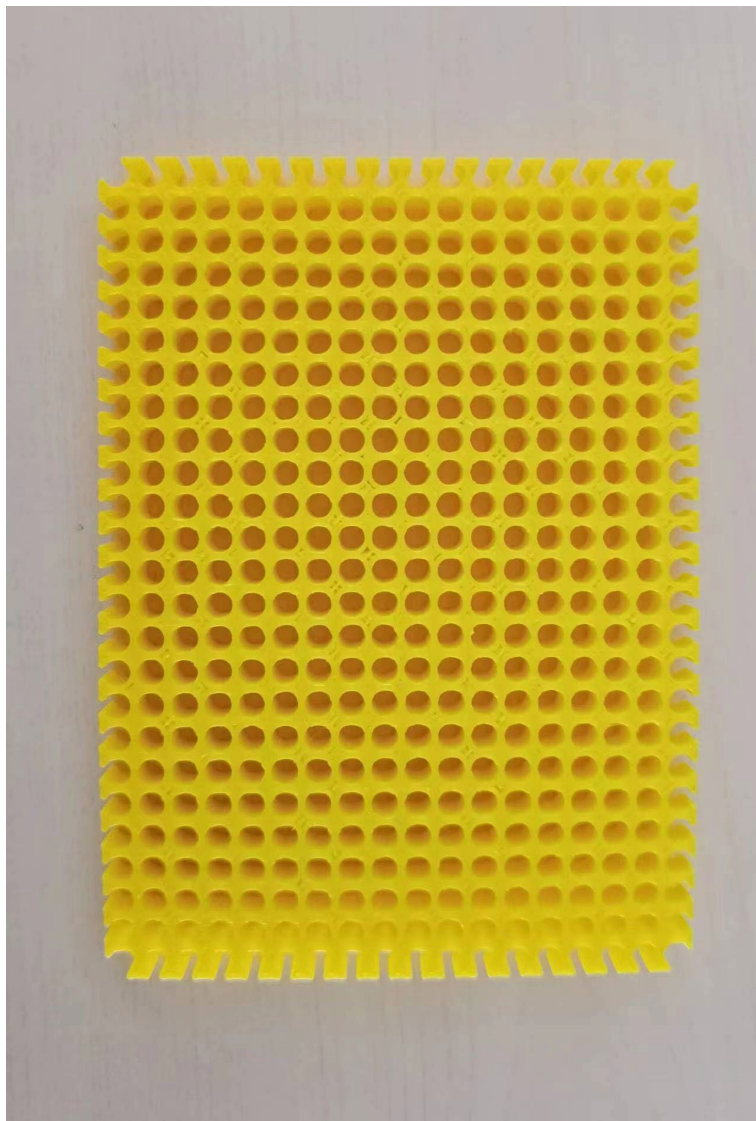
第二次预实验



第三次预实验



第四次预实验



第五次预实验





- 1、在第一次实验中，小球将尺子击飞。
- 2、从第二次实验可以看出尺子发生微小形变。纸张有较小形变。小球被弹起，高度很小。
- 3、在第三、四次实验中，小球将尺子击飞
- 3、在第五次实验中，尺子没能承受住小球。

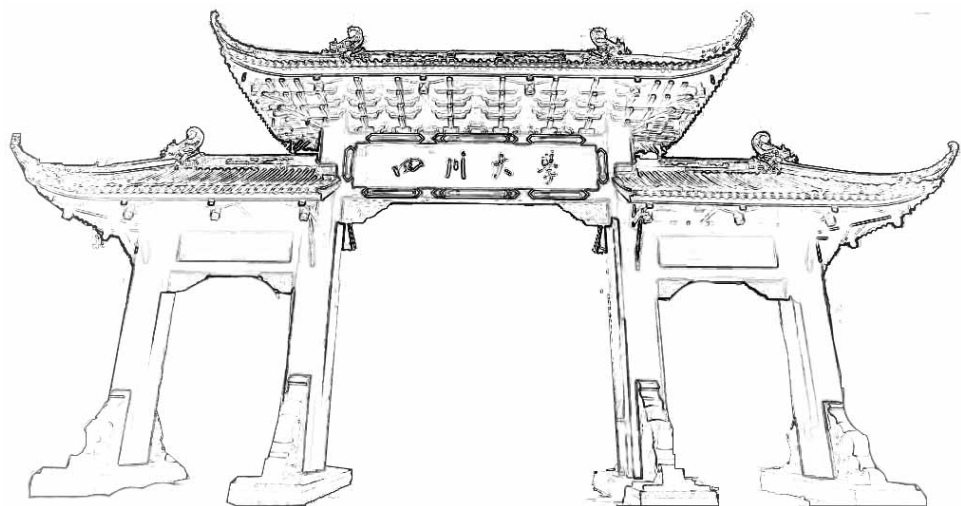


由预实验，有以下发现：

- 1、推测是大气压强稳住了尺子。
- 2、成功的实验中纸均发生了可见的微小形变。



我们不难发现纸张并没有形成完全密封的空间，与桌面间留有空气进出的缝隙。发生形变后缝隙进一步扩大

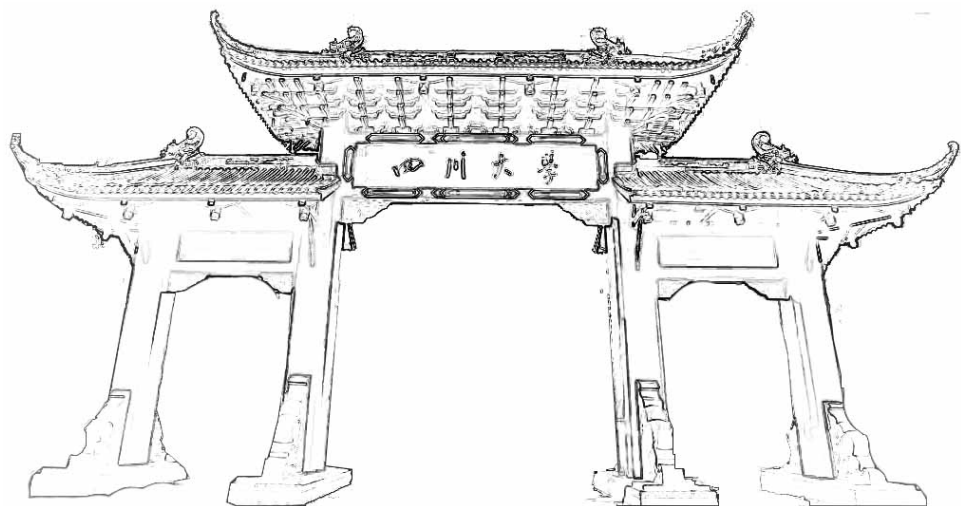


参数分析



相关的参数：

- 1、尺子方面：尺子厚度、长度、约束端长度、杨氏模量、泊松比
- 2、环境方面：大气压、温度、湿度、气体黏度
- 3、小球方面：质量、释放高度



理论构建



一、建立模型

根据实验录像，翘起的约束端造成了纸张的微小形变。
从预实验我们做出以下的假设：

在尺子被击中时，约束端快速上翘，将纸中心抬高。
快速增大了纸下空气腔的体积，外部气体流入速率不足以补偿体积增加导致的气压降低形成压力差。



根据预实验中的现象，我们对模型做出以下理想简化：

- 忽略空气阻力造成的能量损失。
- 将尺子视作刚体。
- 将空气视为理想气体，将空气流动视为黏性低雷诺数层流。
- 根据仿真结果，将空气腔边界的压强分布视为线性变化，内部视为压强均匀。
- 根据大量实验录像，将空气腔近似为开口面为底、纸张宽度的柱体。
- 忽略纸和尺子的重力和它们之间的摩擦。



在以上控制方程中，主要有两组重要的参数。

◆ 动力学：决定 $\frac{dV}{dt}$ 。球释放高度、尺子泊松比、长度、约束长度、杨氏模量。

◆ 流体力学：决定 D 与 $\frac{\partial p}{\partial x}$ 。外界压强、黏度、温度、湿度
这两组参数相对独立。



一、尺子动力学描述

对尺子分析，一个质量为 m 的球从 h 高度释放，给质量为 M ，长度为 L 尺子一个脉冲。设小球动能转化为尺子动能效率为 k ，纸对尺子的作用力为 F_l ，尺子转过角度为 θ 。对尺子中点（支撑点）分析，

$$\int_0^T \frac{L}{2} F_l \cos \theta dt = L \sqrt{\frac{k}{6} m M g h}$$

令碰撞后尺子初始角动量为 J 。



二、空气动力学分析

对纸张底部空气腔分析，体积为 V ，压强为 p' ，外界大气压为 p ，开始时体积为 V_0 。由实际气体状态方程，气体扩散定理

$$p' = \left(p + \frac{An_0^2}{V_0^2} \right) \frac{pV_0 + \iint_S \frac{D\partial p}{\partial x} ds dt}{n_0} \frac{V_0 - n_0 B}{V - (pV_0 + \iint_S \frac{D\partial p}{\partial x} ds dt)B} - \frac{A(pV_0 + \iint_S \frac{D\partial p}{\partial x} ds dt)^2}{V^2}$$

其中， l 是纸的宽度， D 是扩散系数。



由斯托克斯-爱因斯坦关系，扩散系数D等于。

$$D = \frac{k_B T}{6\pi\eta r}$$

其中， k_B 是玻尔兹曼常量， T 是绝对温度， η 是气体黏度， r 是分子平均半径（取 $3.8 \times 10^{-10} \text{m}$ ）

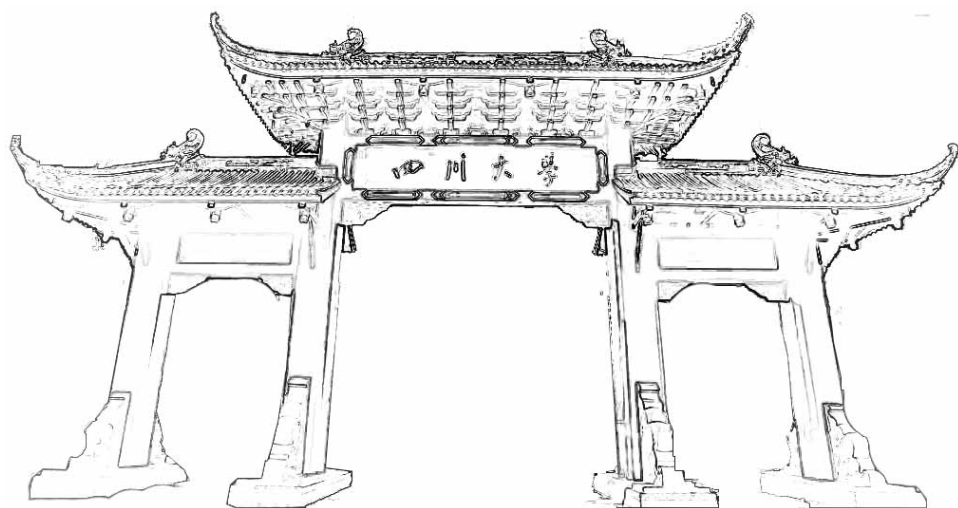


在 $0^{\circ}\text{C} - 300^{\circ}\text{C}$ ， $0-50\text{MPa}$ 内，空气黏度与温度、压强近似符合以下关系式 $\eta = f(t, p) =$

$$\begin{aligned} & c_1 + c_2 t + c_3 p + c_4 t^2 + c_5 t p + c_6 p^2 + c_7 t^3 + \\ & c_8 t^2 p + c_9 t p^2 + c_{10} p^3 + c_{11} t^4 + c_{12} t^3 p + \\ & c_{13} t^2 p^2 + c_{14} t p^3 + c_{15} p^4 + c_{16} t^5 + c_{17} t^4 p + \\ & c_{18} t^3 p^2 + c_{19} t^2 p^3 + c_{20} t p^4 + c_{21} p^5, \end{aligned}$$

其系数如下表：

系数	值	系数	值	系数	值
c_1	1.660×10^{-5}	c_8	9.659×10^{-12}	c_{15}	1.262×10^{-12}
c_2	4.691×10^{-8}	c_9	-5.121×10^{-11}	c_{16}	-2.303×10^{-18}
c_3	1.656×10^{-7}	c_{10}	-1.883×10^{-10}	c_{17}	4.086×10^{-17}
c_4	4.983×10^{-12}	c_{11}	1.640×10^{-15}	c_{18}	-8.316×10^{-17}
c_5	-1.453×10^{-9}	c_{12}	-3.227×10^{-14}	c_{19}	-5.593×10^{-16}
c_6	1.088×10^{-8}	c_{13}	9.812×10^{-14}	c_{20}	-4.244×10^{-15}
c_7	-3.774×10^{-13}	c_{14}	7.385×10^{-13}	c_{21}	-1.613×10^{-16}



系统耦合

纸张施加在尺子端点上的力为：

$$F_l = (p - p')S_0$$

对尺子翻转全过程，时间为T，在实验中观察到尺子不能翻越临界角 $\theta_m = 21.8^\circ$ 。因此尺子稳定性条件为

$$\int_0^\theta \left(p - \left(p + \frac{An_0^2}{V_0^2} \right) \frac{pV_0 + \iint_S \frac{D\partial p}{\partial x} dsdt}{n_0} \frac{V_0 - n_0B}{V - (pV_0 + \iint_S \frac{D\partial p}{\partial x} dsdt)B} - \frac{A(pV_0 + \iint_S \frac{D\partial p}{\partial x} dsdt)^2}{V^2} \right) S_0 \cos\theta \frac{L}{2} d\theta = \frac{6J^2}{ML^2}$$

$$\theta \leq \theta_m$$



对 t 时刻,

$$\frac{d\theta}{dt} = \frac{12}{ML^2} \left(J - \int_0^t \left(p - \left(p + \frac{An_0^2}{V_0^2} \right) \frac{pV_0 + \iint_S \frac{D\partial p}{\partial x} ds dt}{n_0} \frac{V_0 - n_0 B}{V - (pV_0 + \iint_S \frac{D\partial p}{\partial x} ds dt)B} - \frac{A(pV_0 + \iint_S \frac{D\partial p}{\partial x} ds dt)^2}{V^2} \right) S_0 \cos\theta \frac{L}{2} dt \right)$$



将空气分为三部分：

1、大气：压强恒为 p

2、边界层：在大气与空气腔之间，空气视为黏性流体，宽度约为1毫米。根据先前假设，压强梯度沿垂直纸张开口面方向且大小为常数。

3、空气腔：视为平衡理想气体。

并且计算过程中，忽略 V_0 ，



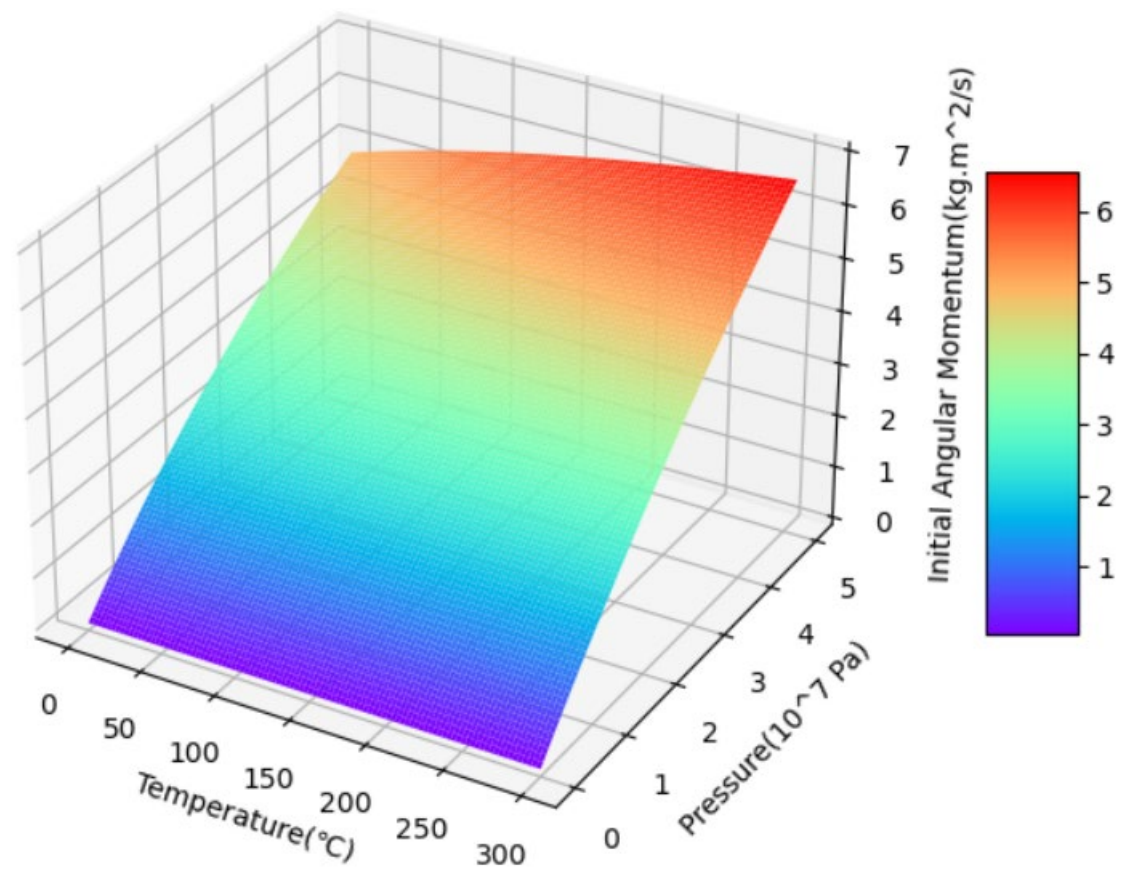
对 t 时刻,

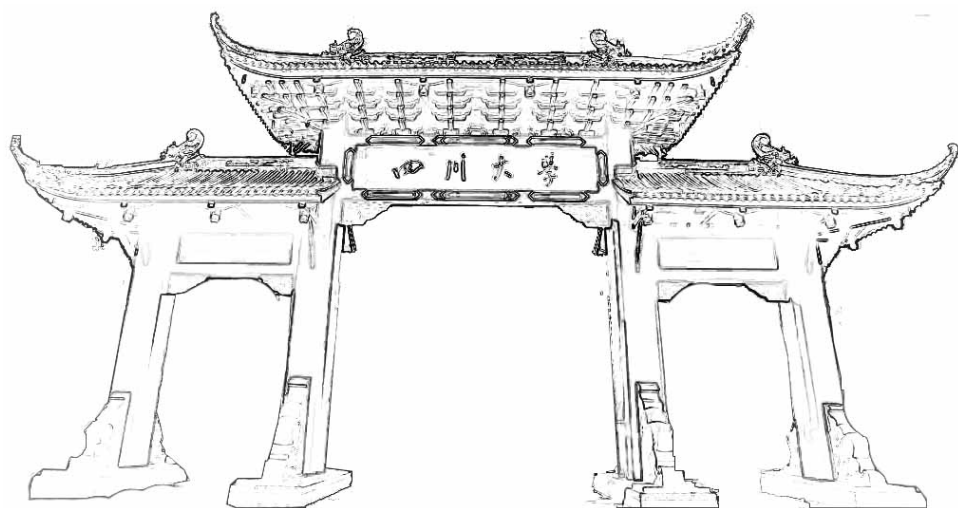
$$\frac{d\theta}{dt} = \frac{12}{ML^2} \left(J - \frac{3S_0 L p l \pi r (c_0 + c_1 T + c_2 p)}{K k_B T} (1 - \exp(-\frac{K k_B T t}{6 l \pi r (c_0 + c_1 T + c_2 p)})) \right)$$

令 $a = \frac{kD}{l}$, 解方程可得:

$$J = \frac{ML^2}{12} \frac{\theta}{t} + \frac{S_0 L p}{2a} (1 - \frac{1}{at} (1 - e^{-at}))$$

令 $\frac{d\theta}{dt} = 0$, 解得临界方程。



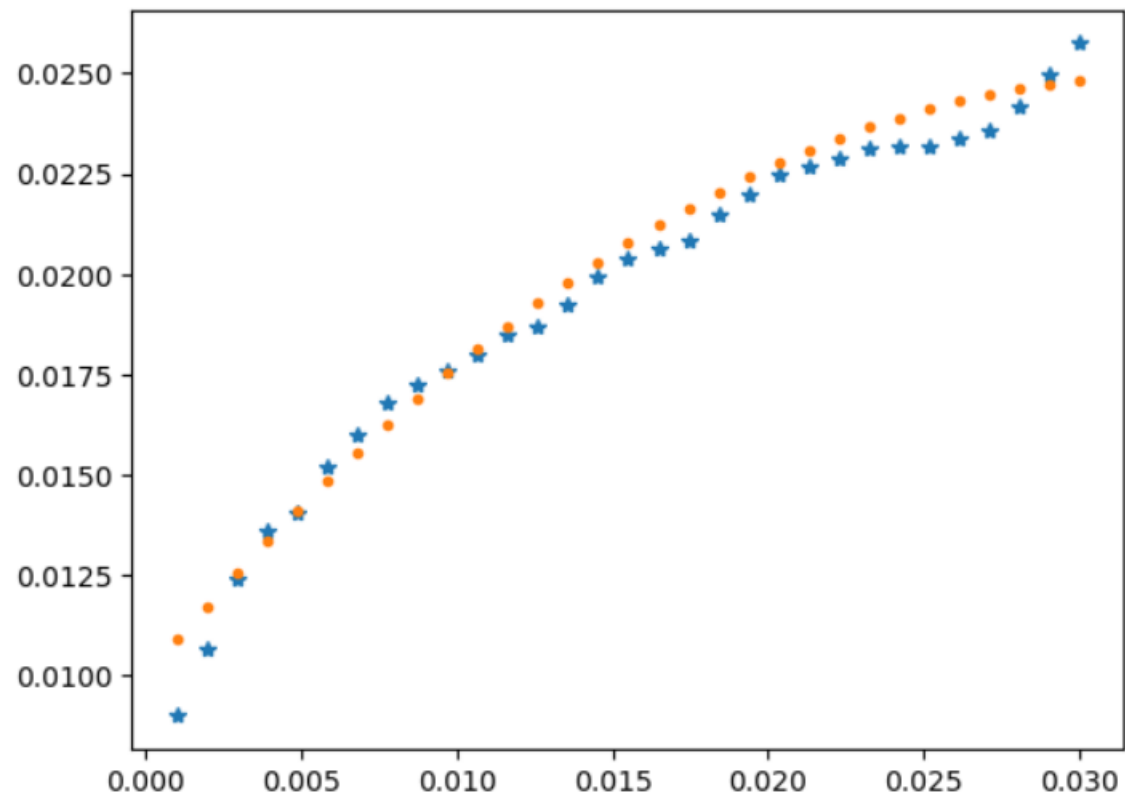


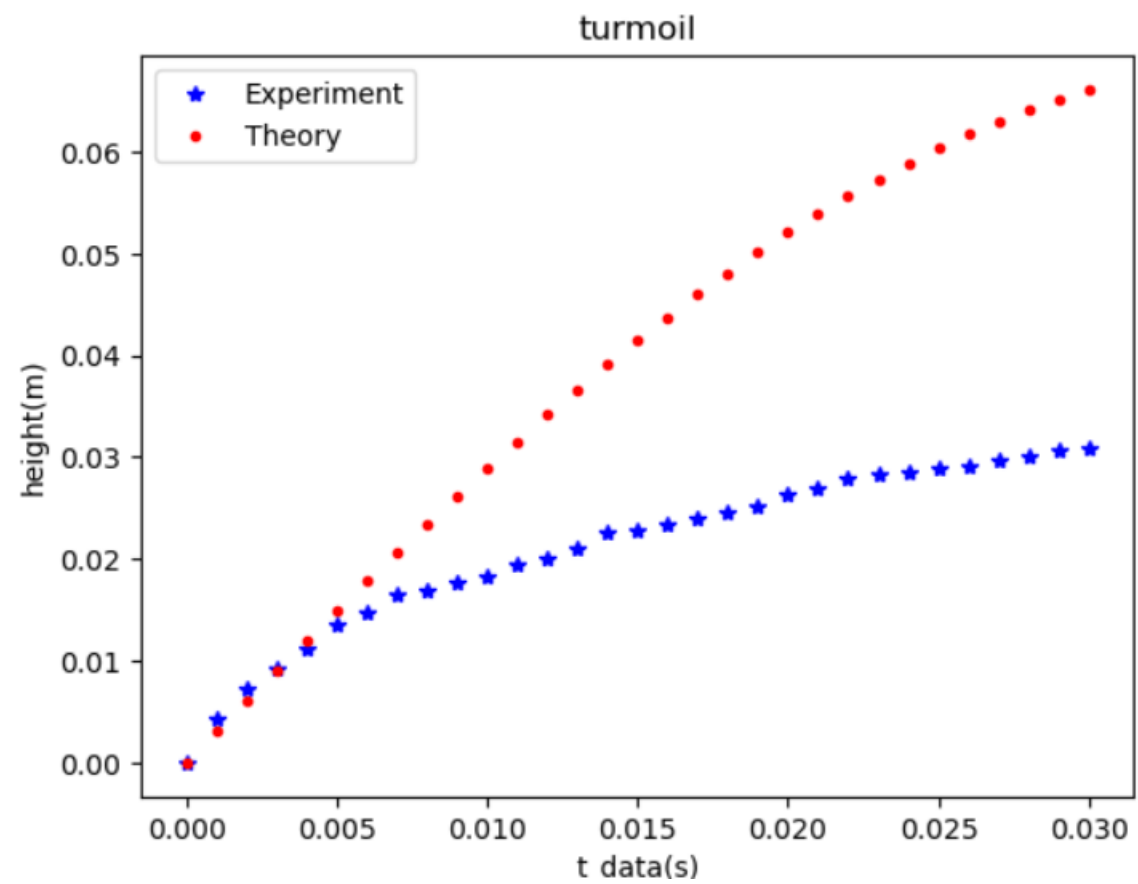
正式实验

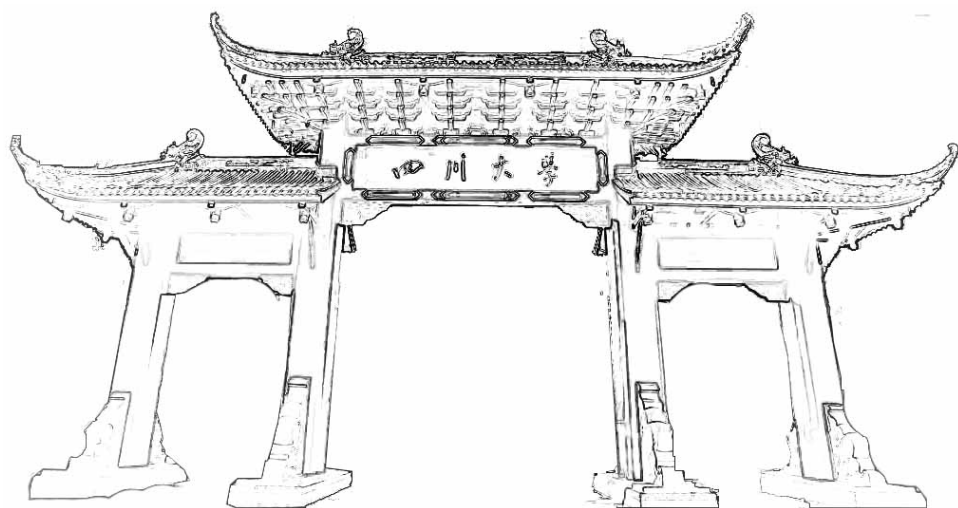


拟合的多项式系数为 $a = -13.88$, $b = 0.91$, $c = 0.01$

Out[18]: [`<matplotlib.lines.Line2D at 0x2bedd17ec50>`]





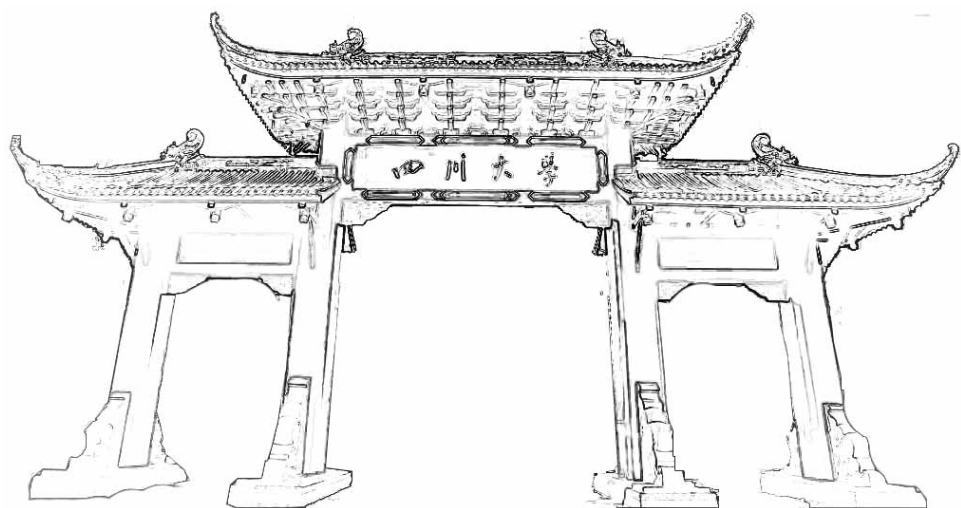


总结



一、尺子不翻是大气压强的作用。由于空气内摩擦不可忽略，气体进入纸底部空气腔速率较慢形成低真空区。纸上下的压强差给纸施加了压力。

二、“尺子把戏”涉及众多参数，本实验着重研究了温度、压强和初始角动量三个参数。其余绝大多数参数，如空气黏度，小球释放高度都可表示为这三个参数的函数。



误差分析



理论误差

- 1、模型误差：忽略了尺子与球的弹性，与现实明显不符。
- 2、将空气流动视为层流且忽略了空气进入腔体后的黏性。
- 3、在不能使用数值方法的符号推导阶段为了可以得到解析解大量使用近似。
- 4、将尺子稳定条件视为不翻过某一临界角
- 5、忽略了桌面与尺子、尺子与纸张、空气的相互作用
- 6、忽略了纸、尺子的重力
- 7、在估算黏度仅将拟合多项式保留至一次项，精度很低。
- 8、处理气压梯度时的近似处理。
- 9、对空气腔体积估算。



实验误差

所有数据均靠Tracker手动打点完成，受拍摄角度和打点精度的影响

实验室开了空调，但室内温度不一定等于设定温度。压强也不一定等于标准大气压。



待改进的部分

- 1、对小球和支持接触部分采取JKR弹性接触模型计算，考虑钢尺的大挠度弯曲。
- 2、引入支持力系数以考虑摩擦力影响。
- 3、研究尺子振动以探究尺子受扰动失稳过程。
- 4、引入更多变量以找到更普适解。
- 5、改变建模思路，采用CFD方法构建流场。
- 6、解决此前受计算资源限制而放弃的数值方法。
- 7、考虑引入PINN物理信息神经网络和ELM极限学习机。
- 8、学习数学知识，增强数学建模能力以改善模型。



实验待改进的部分

在不同的压强、温度下，精确释放小球，得到临界面。



Ivanov, Dragia, and Stefan Nikolov. "Is It Simple to Explain Simple Experiments? The 'heavy Newspaper' Stick Break." *Physics Education* 55.1 (2020): 15014. Web.

Sharma, Bhanuday, Rakesh Kumar, and Savitha Pareek. "Bulk Viscosity of Dilute Gases and Their Mixtures." *Fluids (Basel)* 8.1 (2023): 28. Web.

朱维耀. "考虑压力作用的气体或气体混合物的扩散渗流方程." *Shiyou Kantan Kaifa* 16.6 (1989): 84-87. Web.

邓强国 宋鹏云 毛文元 许恒杰. "气体黏度与温度和压力关系的拟合表达式." *排灌机械工程学报* 35.2 (2017): 144-51. Web.

晋宏营. "计算机模拟估算气体的扩散系数." *Jisuanji Yu Yingyong Huaxue* 29.11 (2012): 1294-298. Web.

兰道, 利夫希茨, and 李俊峰. 力学. 北京: 高等教育出版社, 2007. Print. 理论物理学教程 Li Lun Wu Li Xue Jiao Cheng 1.

兰道, 利夫希茨, and 李植. 流体力学. 北京: 高等教育出版社, 2013. Print. 理论物理学教程 Li Lun Wu Li Xue Jiao Cheng 第6卷.

CHEN, Wei. "Discussions on Brownian motion of colloids", *Physics Experimentation* 38.1 (2018): 29-32.

感谢聆听！



```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# 定义变量
KA = 1.54*10**-4
t_data = np.linspace(0.000, 0.030, 31)

# 定义数据
y = np.array([0.551, 0.565, 0.575, 0.582, 0.588, 0.596, 0.600, 0.606, 0.607, 0.610, 0.612, 0.616, 0.618, 0.621, 0.626, 0.627, 0.629, 0.631,
plt.plot(t_data, y, '*', color='blue', label='Experiment')
plt.xlabel('t_data(s)')
plt.ylabel('height(m)')
plt.title('turmoil')
plt.legend(loc='upper left')

# 定义函数 theta(x)
def theta(x):
    return 21613 * (0.002 * x - 0.00623 * (x - (1 - np.exp(-KA*x)) / KA) / KA)

# 计算 mid
mid = theta(t_data)
mid = pd.DataFrame(mid, dtype=float)

# 计算 y
y = 0.07*np.sin(mid)
# 绘制散点图
plt.plot(t_data, y, '.', color='red', label='Theory')
plt.legend(loc='upper left')

# 显示图像
plt.show()
```



```
import matplotlib.pyplot as plt
import numpy as np

fig = plt.figure(figsize=(12, 6))
ax = fig.add_subplot(111, projection='3d')

X, Y = np.meshgrid(np.linspace(0, 300, 100), np.linspace(0, 50000000, 100))
KA=(9.6*10**-12)*(X+271) / (1.66 * 10**(-5) * np.ones((100, 100)) + (4.7 * 10**-8) * X + (1.66 * 10**-13)* Y)
Z = (5.8 * 10**-4) * np.ones((100, 100)) + (0.06237) *(Y -Y*(np.ones((100,100))-np.exp(KA*-0.03))/(KA*0.03))

surf = ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap=plt.get_cmap('rainbow'))
ax.set_zlim(0,7)
ax.set_xlabel("Temperature(°C)")
ax.set_ylabel("Pressure(10^7 Pa)")
ax.set_zlabel("Initial Angular Momentum(kg.m^2/s)")
fig.colorbar(surf, shrink=0.5, aspect=5)
plt.show()
```



```
import numpy as np
from scipy.optimize import curve_fit
import matplotlib.pyplot as plt

# 定义要拟合的多项式函数
def polynomial_func(t, a, b, c):
    return a * t**2 + b * t + c

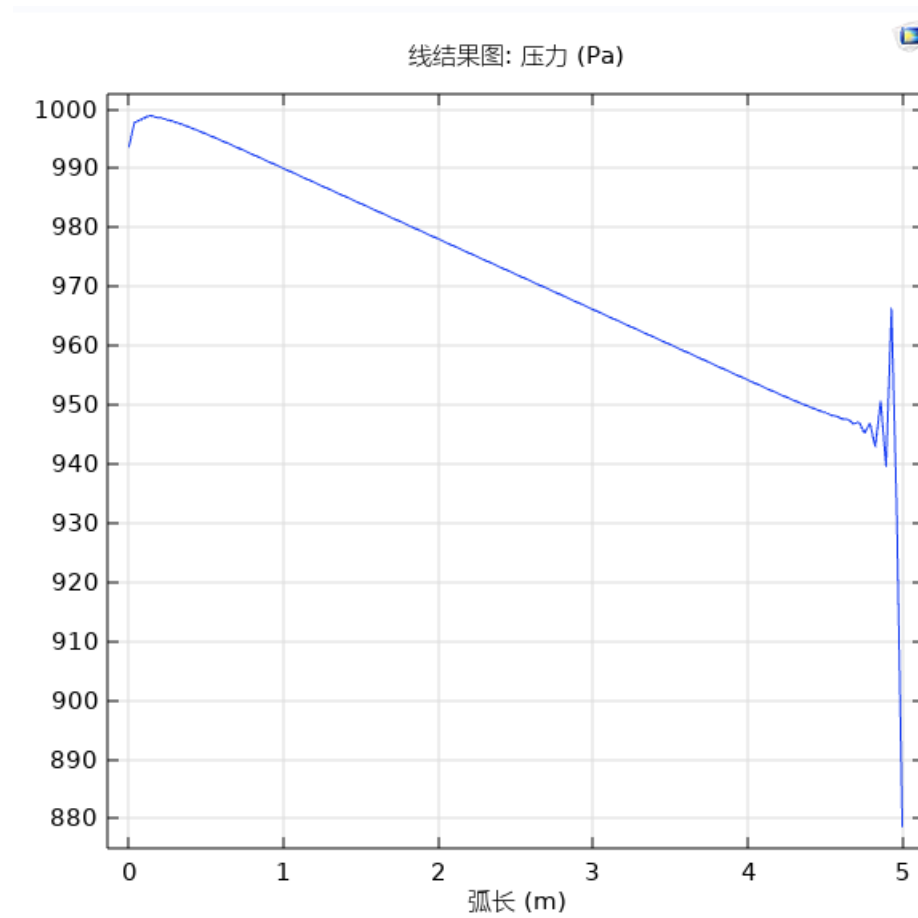
t_data = np.linspace(0.001, 0.036, 36)
y_0 = np.array([0.521, 0.522, 0.527, 0.533, 0.543, 0.552, 0.566, 0.574, 0.582, 0.589, 0.595, 0.601, 0.606, 0.609, 0.612, 0.614, 0.615, 0.616, 0.617, 0.618, 0.619, 0.62, 0.621, 0.622, 0.623, 0.624, 0.625, 0.626, 0.627, 0.628, 0.629, 0.63, 0.631, 0.632, 0.633, 0.634, 0.635, 0.636])
y_data = y_0 - 0.51 * np.ones(36)

# 使用最小二乘法拟合多项式
params, covariance = curve_fit(polynomial_func, t_data, y_data)

# 输出拟合的参数
a_fit, b_fit, c_fit = params
print("拟合的多项式系数为 a = {:.2f}, b = {:.2f}, c = {:.2f}".format(a_fit, b_fit, c_fit))
plt.plot(t_data, y_data, '*')
def g(x):
    return -123.60 * x**2 + 8.44 * x
y = g(t_data)
plt.plot(t_data, y, '.')
```




质量_A	质量_B [←]	质量_C [←]	质量_D [←]	质量_E [←]	质量_F [←]	质量_G [←]	质量_H [←]	质量_I [←]	质量_J [←]
y [←]	y [←]	y [←]	y [←]	y [←]	y [←]	y [←]	y [←]	y [←]	y [←]
4.849689365E-1 [←]	5.003346309E-1 [←]	5.243327377E-1 [←]	5.403890250E-1 [←]	5.506124723E-1 [←]	5.505752718E-1 [←]	5.378856254E-1 [←]	5.216487188E-1 [←]	5.004017907E-1 [←]	4.835507787E-1 [←]
4.866954190E-1 [←]	5.020611134E-1 [←]	5.245053860E-1 [←]	5.431513971E-1 [←]	5.594032252E-1 [←]	5.653366973E-1 [←]	5.495393823E-1 [←]	5.304405512E-1 [←]	5.058966859E-1 [←]	4.850160841E-1 [←]
4.870407155E-1 [←]	5.020611134E-1 [←]	5.262318685E-1 [←]	5.479855481E-1 [←]	5.658131492E-1 [←]	5.749186752E-1 [←]	5.630059459E-1 [←]	5.399650362E-1 [←]	5.139558655E-1 [←]	4.886793476E-1 [←]
4.868680673E-1 [←]	4.998166861E-1 [←]	5.277857027E-1 [←]	5.516111614E-1 [←]	5.720399326E-1 [←]	5.821699017E-1 [←]	5.697392277E-1 [←]	5.509548266E-1 [←]	5.198170871E-1 [←]	4.894120003E-1 [←]
4.860048260E-1 [←]	4.998166861E-1 [←]	5.289942405E-1 [←]	5.557547194E-1 [←]	5.769847311E-1 [←]	5.878672940E-1 [←]	5.754366200E-1 [←]	5.494895213E-1 [←]	5.187181081E-1 [←]	4.850160841E-1 [←]
4.860048260E-1 [←]	4.991260931E-1 [←]	5.296848335E-1 [←]	5.585170914E-1 [←]	5.828452330E-1 [←]	5.961544101E-1 [←]	5.842416808E-1 [←]	5.586476799E-1 [←]	5.201834135E-1 [←]	4.864813895E-1 [←]
4.860048260E-1 [←]	4.979175553E-1 [←]	5.329651503E-1 [←]	5.611068151E-1 [←]	5.852260619E-1 [←]	6.000389957E-1 [←]	5.907159902E-1 [←]	5.648752279E-1 [←]	5.220150452E-1 [←]	4.875803685E-1 [←]
4.860048260E-1 [←]	4.982628518E-1 [←]	5.324472055E-1 [←]	5.633512424E-1 [←]	5.899877198E-1 [←]	6.059953604E-1 [←]	5.984851614E-1 [←]	5.707364494E-1 [←]	5.264109614E-1 [←]	4.864813895E-1 [←]
4.860048260E-1 [←]	4.973996106E-1 [←]	5.345189845E-1 [←]	5.652503732E-1 [←]	5.921854080E-1 [←]	6.070312499E-1 [←]	6.002979681E-1 [←]	5.754986919E-1 [←]	5.260446350E-1 [←]	4.868477158E-1 [←]
4.860048260E-1 [←]	4.973996106E-1 [←]	5.345189845E-1 [←]	5.671495039E-1 [←]	5.941999556E-1 [←]	6.098799460E-1 [←]	6.021107747E-1 [←]	5.773303237E-1 [←]	5.267772877E-1 [←]	4.868477158E-1 [←]
4.860048260E-1 [←]	4.958457763E-1 [←]	5.360728188E-1 [←]	5.692212829E-1 [←]	5.963976438E-1 [←]	6.124696698E-1 [←]	6.054774156E-1 [←]	5.817262398E-1 [←]	5.260446350E-1 [←]	4.868477158E-1 [←]
4.860048260E-1 [←]	4.958457763E-1 [←]	5.371087083E-1 [←]	5.707751172E-1 [←]	5.991447541E-1 [←]	6.158363107E-1 [←]	6.088440565E-1 [←]	5.831915452E-1 [←]	5.282425931E-1 [←]	4.868477158E-1 [←]
4.860048260E-1 [←]	4.944645903E-1 [←]	5.365907635E-1 [←]	5.738827857E-1 [←]	6.009761609E-1 [←]	6.179080897E-1 [←]	6.093620012E-1 [←]	5.842905243E-1 [←]	5.286089194E-1 [←]	4.868477158E-1 [←]
4.860048260E-1 [←]	4.920475148E-1 [←]	5.371087083E-1 [←]	5.740554340E-1 [←]	6.031738492E-1 [←]	6.210157582E-1 [←]	6.142824764E-1 [←]	5.890527668E-1 [←]	5.319058566E-1 [←]	4.890456739E-1 [←]
4.860048260E-1 [←]	4.915295701E-1 [←]	5.355548740E-1 [←]	5.764725095E-1 [←]	6.051883967E-1 [←]	6.256772610E-1 [←]	6.202388410E-1 [←]	5.949139883E-1 [←]	5.348364674E-1 [←]	4.890456739E-1 [←]
4.860048260E-1 [←]	4.904936806E-1 [←]	5.345189845E-1 [←]	5.778536955E-1 [←]	6.072029443E-1 [←]	6.269721228E-1 [←]	6.225695924E-1 [←]	5.989435782E-1 [←]	5.406976889E-1 [←]	4.890456739E-1 [←]
4.860048260E-1 [←]	4.892851428E-1 [←]	5.357275223E-1 [←]	5.776810472E-1 [←]	6.079355070E-1 [←]	6.287849295E-1 [←]	6.254182886E-1 [←]	6.022405153E-1 [←]	5.406976889E-1 [←]	4.890456739E-1 [←]
4.860048260E-1 [←]	4.882492533E-1 [←]	5.346916328E-1 [←]	5.776810472E-1 [←]	6.090343511E-1 [←]	6.305977361E-1 [←]	6.285259571E-1 [←]	6.062701051E-1 [←]	5.399650362E-1 [←]	4.890456739E-1 [←]
4.860048260E-1 [←]	4.875586603E-1 [←]	5.345189845E-1 [←]	5.776810472E-1 [←]	6.104994766E-1 [←]	6.331874599E-1 [←]	6.321515704E-1 [←]	6.077354105E-1 [←]	5.516874793E-1 [←]	4.897783266E-1 [←]
4.860048260E-1 [←]	4.875586603E-1 [←]	5.382972172E-1 [←]	5.788895850E-1 [←]	6.130634462E-1 [←]	6.350002665E-1 [←]	6.355182112E-1 [←]	6.102996949E-1 [←]	5.527864584E-1 [←]	4.901446530E-1 [←]
4.860048260E-1 [←]	4.875586603E-1 [←]	5.382972172E-1 [←]	5.800981227E-1 [←]	6.165431192E-1 [←]	6.386258798E-1 [←]	6.391438245E-1 [←]	6.139629584E-1 [←]	5.535191111E-1 [←]	4.908773057E-1 [←]
4.860048260E-1 [←]	4.875586603E-1 [←]	5.348362407E-1 [←]	5.800981227E-1 [←]	6.165431192E-1 [←]	6.414745759E-1 [←]	6.425104654E-1 [←]	6.183588746E-1 [←]	5.524201320E-1 [←]	4.908773057E-1 [←]
4.860048260E-1 [←]	4.875586603E-1 [←]	5.325289230E-1 [←]	5.787169367E-1 [←]	6.167262599E-1 [←]	6.440642997E-1 [←]	6.458771063E-1 [←]	6.212894854E-1 [←]	5.535191111E-1 [←]	4.912436320E-1 [←]
4.860048260E-1 [←]	4.910116253E-1 [←]	5.297601418E-1 [←]	5.762998612E-1 [←]	6.167262599E-1 [←]	6.451001892E-1 [←]	6.479488853E-1 [←]	6.256854015E-1 [←]	5.568160482E-1 [←]	4.912436320E-1 [←]
4.860048260E-1 [←]	4.910116253E-1 [←]	5.288372147E-1 [←]	5.745733787E-1 [←]	6.170925413E-1 [←]	6.463950510E-1 [←]	6.479488853E-1 [←]	6.282496860E-1 [←]	5.568160482E-1 [←]	4.912436320E-1 [←]
4.860048260E-1 [←]	4.910116253E-1 [←]	5.276835559E-1 [←]	5.730195445E-1 [←]	6.156274158E-1 [←]	6.471719682E-1 [←]	6.495027196E-1 [←]	6.297149914E-1 [←]	5.568160482E-1 [←]	4.912436320E-1 [←]
4.860048260E-1 [←]	4.910116253E-1 [←]	5.267606288E-1 [←]	5.714657102E-1 [←]	6.137960089E-1 [←]	6.484668301E-1 [←]	6.518334709E-1 [←]	6.319129494E-1 [←]	5.626772698E-1 [←]	4.912436320E-1 [←]
4.860048260E-1 [←]	4.927381078E-1 [←]	5.244533111E-1 [←]	5.692212829E-1 [←]	6.123308835E-1 [←]	6.495027196E-1 [←]	6.554590842E-1 [←]	6.333782548E-1 [←]	5.645089015E-1 [←]	4.952732218E-1 [←]
4.860048260E-1 [←]	4.934287008E-1 [←]	5.242225793E-1 [←]	5.669768557E-1 [←]	6.101331952E-1 [←]	6.505386091E-1 [←]	6.572718908E-1 [←]	6.381404974E-1 [←]	5.758650183E-1 [←]	5.004017907E-1 [←]
4.860048260E-1 [←]	4.941192938E-1 [←]	5.235303840E-1 [←]	5.643871319E-1 [←]	6.075692256E-1 [←]	6.526103881E-1 [←]	6.619333936E-1 [←]	6.429027399E-1 [←]	5.842905243E-1 [←]	5.095599494E-1 [←]
4.860048260E-1 [←]	4.949825351E-1 [←]	5.221459934E-1 [←]	5.612794634E-1 [←]	6.064703815E-1 [←]	6.539052500E-1 [←]	6.624513384E-1 [←]	6.494966141E-1 [←]	5.952803147E-1 [←]	5.242130033E-1 [←]





$$\text{令 } \frac{d\theta}{dt} = 0, \text{ 得 } J - \int_0^t \left(p - \frac{pV_0 + \iint_S \frac{D\partial p}{\partial x} ds dt}{V} \right) S_0 \cos\theta \frac{L}{2} dt = 0。$$

由先前假设，气压呈线性变化，我们有：

$$p' = \frac{pV_0 + DKS(pt - \int p' dt)}{V}$$

令 $Q = \int p' dt$ ，整理：

$$\frac{dQ}{dt} + \frac{kSD}{V} Q = \frac{pV_0 + kSDPt}{V}$$

忽略 V_0 ，利用假设， $V = Sl$ ，解微分方程。

$$Q = p \left(t - \frac{1}{a} + \frac{1}{ae^{at}} \right)$$

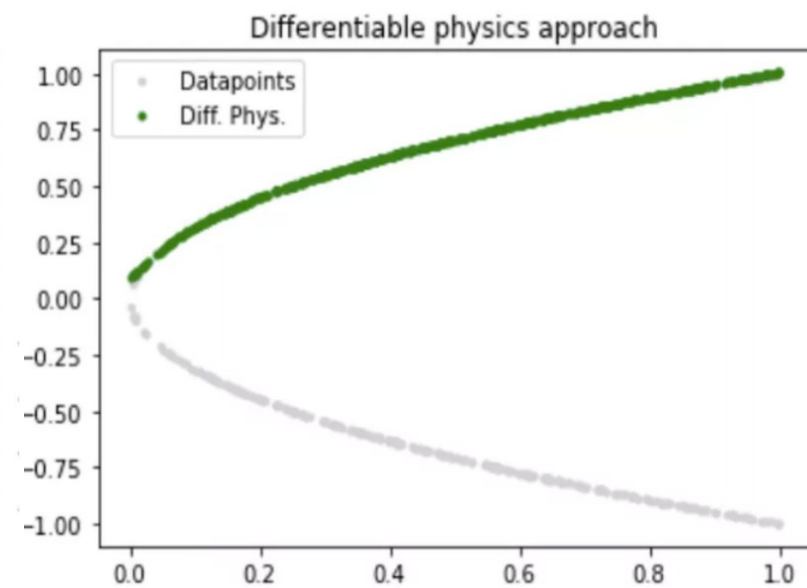
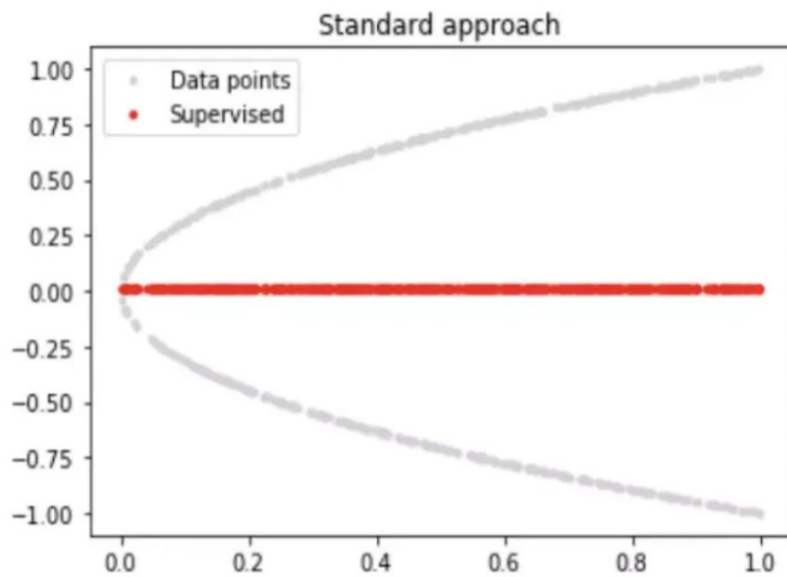
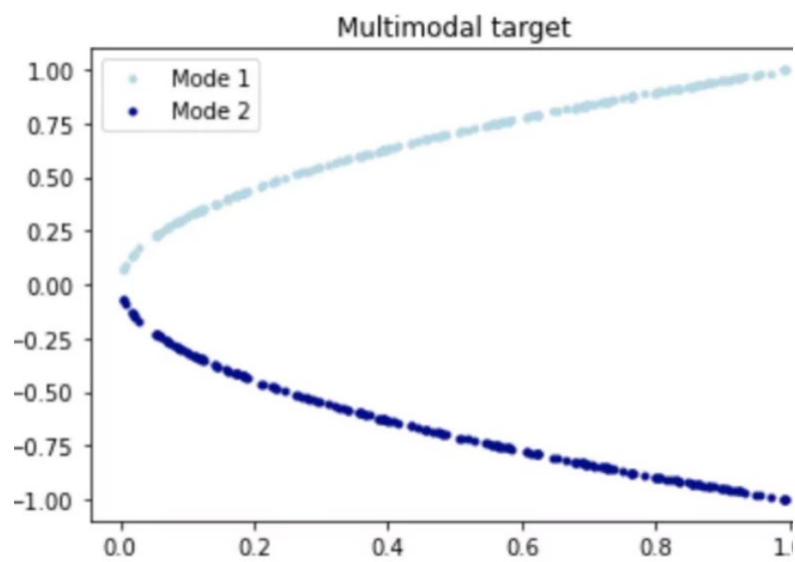


对 t 时刻,

$$\frac{d\theta}{dt} = \frac{12}{ML^2} \left(J - \frac{S_0 L p}{2a} (1 - e^{-at}) \right)$$
$$\theta = \frac{12}{ML^2} \left(Jt - \frac{S_0 L p}{2a} \left(t - \frac{1}{a} (1 - e^{-at}) \right) \right)$$

即:

$$\theta(t) = \frac{12}{ML^2} \left(Jt - \frac{3S_0 L p l \pi r (c_0 + c_1 T + c_2 p)}{K k_B T} \left(t - \frac{6l \pi r (c_0 + c_1 T + c_2 p)}{K k_B T} (1 - \exp(-\frac{K k_B T t}{6l \pi r (c_0 + c_1 T + c_2 p)})) \right) \right)$$





```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np

class PINN(nn.Module):
    def __init__(self):
        super(PINN, self).__init__()
        self.fc1 = nn.Linear(2, 50)
        self.fc2 = nn.Linear(50, 50)
        self.fc3 = nn.Linear(50, 1)

    def forward(self, x):
        x = torch.sin(self.fc1(x))
        x = torch.sin(self.fc2(x))
        x = self.fc3(x)
        return x

# 定义方腔流体N-S方程的PINN求解器类
class NSPINNSolver:
    def __init__(self, x, y, u, v, nu, device='cpu'):
        self.device = device
```

```
self.x = torch.tensor(x, dtype=torch.float32, device=device, requires_grad=True)
self.y = torch.tensor(y, dtype=torch.float32, device=device, requires_grad=True)
self.u = torch.tensor(u, dtype=torch.float32, device=device, requires_grad=True)
self.v = torch.tensor(v, dtype=torch.float32, device=device, requires_grad=True)
self.t = torch.tensor(t, dtype=torch.float32, device=device, requires_grad=True)
self.px = torch.tensor(px, dtype=torch.float32, device=device, requires_grad=True)
self.py = torch.tensor(py, dtype=torch.float32, device=device, requires_grad=True)

self.model = PINN().to(device)
self.optimizer = optim.Adam(self.model.parameters(), lr=0.001)
self.loss_fn = nn.MSELoss()

self.nu = nu
self.rho = rho

ages
net_NS(self, x, y):
    inputs = torch.cat((x, y), dim=1)
    outputs = self.model(inputs)
    u = outputs[:, 0:1]
    v = outputs[:, 1:2]
    return u, v
```



```
px, py = self.net_NS(self.x, self.y)
rho = input(...)

u_x = torch.autograd.grad(u.sum(), self.x, create_graph=True, retain_graph=True)[0]
u_y = torch.autograd.grad(u.sum(), self.y, create_graph=True, retain_graph=True)[0]
u_xx = torch.autograd.grad(u_x.sum(), self.x, create_graph=True, retain_graph=True)[0]
u_yy = torch.autograd.grad(u_y.sum(), self.y, create_graph=True, retain_graph=True)[0]
u_t = torch.autograd.grad(u.sum(), self.y, create_graph=True, retain_graph=True)[0]

v_x = torch.autograd.grad(v.sum(), self.x, create_graph=True, retain_graph=True)[0]
v_y = torch.autograd.grad(v.sum(), self.y, create_graph=True, retain_graph=True)[0]
v_xx = torch.autograd.grad(v_x.sum(), self.x, create_graph=True, retain_graph=True)[0]
v_yy = torch.autograd.grad(v_y.sum(), self.y, create_graph=True, retain_graph=True)[0]
v_t = torch.autograd.grad(v_y.sum(), self.y, create_graph=True, retain_graph=True)[0]

p_x = torch.autograd.grad(px.sum(), self.y, create_graph=True, retain_graph=True)[0]
p_y = torch.autograd.grad(py.sum(), self.y, create_graph=True, retain_graph=True)[0]

q_x = torch.autograd.grad(u_x.sum() + v_y.sum(), self.x, create_graph=True, retain_graph=True)[0]
q_y = torch.autograd.grad(u_y.sum() + v_y.sum(), self.y, create_graph=True, retain_graph=True)[0]

eq1 = u_xx + u_yy + q_x
eq2 = v_xx + v_yy + q_y
```



```
        return self.loss_fn(u_b, torch.zeros_like(u_b)) + self.loss_fn(v_b, torch.zeros_like(v_b))
```

```
1 usage
```

```
def solve(self, epochs=10000):
```

```
    for epoch in range(epochs):
```

```
        self.optimizer.zero_grad()
```

```
        loss = self.loss_eq() + self.loss_bc()
```

```
        loss.backward()
```

```
        self.optimizer.step()
```

```
    u_pred, v_pred = self.net_NS(self.x, self.y)
```

```
    return u_pred, v_pred
```

```
# 输入数据
```

```
x = np.linspace(0, 1, 51)
```

```
y = np.linspace(0, 1, 51)
```

```
x, y = np.meshgrid(x, y)
```

```
u = np.zeros_like(x)
```

```
v = np.zeros_like(y)
```

```
# 设置参数
```

```
nu = 0.1
```

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

```
# 创建并求解PINN
```

```
solver = NSPINNSolver(x, y, u, v, nu, device='device')
```

```
u_pred, v_pred = solver.solve()
```

```
# 打印结果
```

```
print("u_pred: \n", u_pred)
```

```
print("v_pred: \n", v_pred)
```




使用计算机试算，根据数量级，将方程化简为

$$\int p' dt = \frac{-1388at^3 + 91t^2 + a}{b}$$

Out[766]/NumberForm=
 $-1.675633818 \times 10^{-15}$

Out[767]/NumberForm=
 $2.116208213 \times 10^{-24}$

Out[768]/NumberForm=
 $-5.447595691 \times 10^{-15}$

Out[769]/NumberForm=
 $-3.166557644 \times 10^{-15}$

Out[770]/NumberForm=
 $4.586621342 \times 10^{-24}$

Out[771]/NumberForm=
 $1.781748556 \times 10^{-33}$

Out[772]/NumberForm=
 $8.158527784 \times 10^{-17}$

Out[773]/NumberForm=
 $1.333046879 \times 10^{-24}$

Out[774]/NumberForm=
 $-1.93086056 \times 10^{-33}$

Out[775]/NumberForm=
 $7.500745665 \times 10^{-43}$



对方程（1）化简，可得：

$$p' + \frac{RKTP}{l} \int p' dt = \frac{PV_0 + KSDPt}{Sl}$$

令 $a = kSDP$, $b = \frac{KP}{l}$;

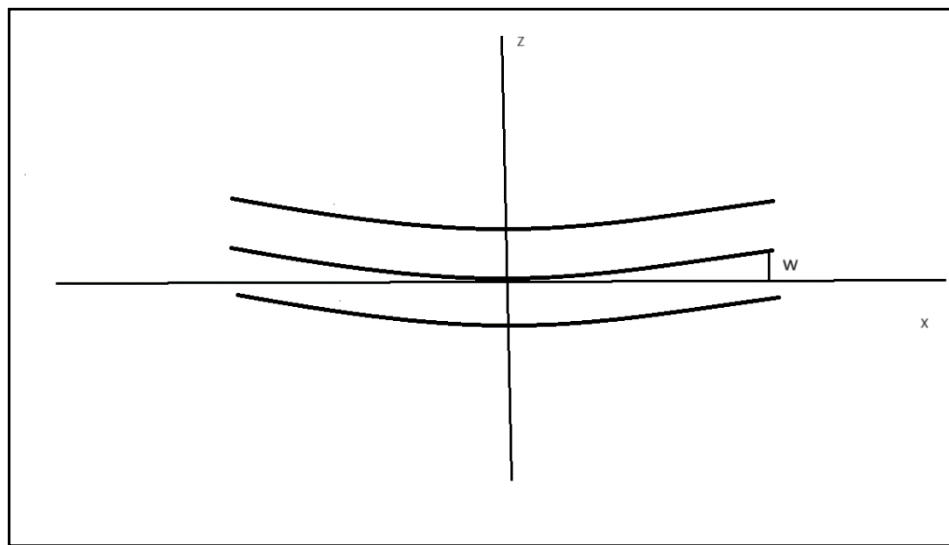
解方程得：

$$\begin{aligned} \int p' dt &= e^{-bt} \left\{ \left(-\frac{1388ae^{bt}t^3}{b} \right) - \frac{((1344972 - 9100000000a)b^3 - 416400000000lRKTSDb^2)e^{bt}t^2}{100000000b^4} \right. \\ &\quad \left. - \frac{(((-10^8a) - 88179)b^3 + (1.82 \times 10^{10}a - 2689944)b^2 + 8.328 \times 10^{11}ab)t}{10^8b^4} \right. \\ &\quad \left. + \frac{969e^{bt}}{100000000b} - \frac{(1000000000a + 88179)e^{bt}}{100000000b^2} - \frac{(2689944 - 18200000000a)e^{bt}}{100000000b^3} + \frac{8328ae^{bt}}{b^4} \right\} \end{aligned}$$



一、建模

建立如图所示坐标系，取中性面上、尺子长度中点为坐标原点， z 轴指向中性面的法线方向，而 xy 平面与未形变的尺子面重合。我们用 w 表示变形后中性面上点的垂直位移，即它们的 z 坐标，也即挠度。





二、模型说明

- 板弯曲时，板内的一些地方发生拉伸，另一些地方发生压缩。显然，在板的凸面发生拉伸，随着向板内深入，拉伸逐渐地减小，最终为零，在后面的板层，压缩逐渐增加。因此，在板的内部存在着一个中性面。在中性面上不存在拉伸或压缩，而在中性面的两边，形变具有相反的符号。并且我们将尺子视作薄板，这个中性面位于尺子厚的中间。
- 因为我们忽略了板的扭转、旋转、水平位移，尺子中这些点在 x 、 y 方向上的位移均为0
- 与尺子内应力相比，小球施加在尺子上的力很小。
- 因为钢尺弯曲很小，我们认为中性面法向矢量 $\mathbf{n}$ 仍沿 z 轴正方向。



三、弯曲板的能量

在弹性体表面的平衡边界条件中，

$$\sigma_{ik}n_k = p_k$$

应力张量在各个面沿 z 方向上的分量都远小于其他分量。即：

$$\sigma_{xz} = \sigma_{yz} = \sigma_{zz} = 0$$

根据应力张量与应变张量的一般公式，我们有：

$$\sigma_{zx} = \frac{E}{1+\sigma} u_{zx}, \sigma_{zy} = \frac{E}{1+\sigma} u_{zy}, \sigma_{zz} = \frac{E}{(1+\sigma)(1-2\sigma)} [(1-\sigma)u_{zz} + \sigma(u_{xx} + u_{yy})]$$

用 w 代替应变张量中的 u_z ，得出所有分量。利用一般公式计算体积自由能，对子的整个体积积分。得到总自由能 F 。



自由能 F 的表达式为：

$$F = \frac{Eh^3}{24(1-\sigma^2)} \iint \left\{ \left(\frac{\partial^2 w}{\partial x^2} + \frac{\partial^2 w}{\partial y^2} \right)^2 + 2(1-\sigma) \left[\left(\frac{\partial^2 w}{\partial x \partial y} \right)^2 - \frac{\partial^2 w}{\partial x^2} \frac{\partial^2 w}{\partial y^2} \right] \right\} dx dy$$

其中，抗弯刚度 $D = \frac{Eh^3}{24(1-\sigma^2)}$ ；

同时，由能量守恒，令 $w|_{x=\frac{l}{2}} = w_0$, $F = mg(w_0 + h)$



四、板的平衡方程

将整个尺子的弯曲过程视作一系列准静态的间断过程。因为当尺子形变确定时，尺子内的应力及它对小球的反作用力也随之确定，所以我们并不关心此时尺子的不平衡性，而将它们作为平衡态处理。为了得出尺子的平衡方程，令自由能函数取极小值，即计算表达式的变分。我们要求自由能变分表达式中 $\delta F + \delta U = 0$ ，而尺子的势能由外力，或者说单位面积上尺子对外界的反作用力 $\mathbf{P}$ 决定。由于挠度极小，我们近似认为力始终竖直向上。因此我们得到条件方程：

$$\delta F + \int \mathbf{P} \delta w ds = 0$$

其中 ds 是面元。



式子左端包含面积分：

$$\int (D\nabla^4 w - P)\delta w ds = 0$$

在这个面积分中 δw 是任意的，那么其系数必为0.因此板的平衡方程是：

$$D\nabla^4 w = P$$



五、动力学描述

根据达朗贝尔原理，我们对运动中的尺子加上惯性力后，作用其上的的是一个平衡力系。设速度和加速度场为 $u_i(x_j, t), a_i(x_j, t)$ ，惯性力为 $-\rho a_i$ 。



六、模型简化

我们注意到，小球施加在尺子上力仅作用于一个点上。实际上尺子的弯曲应当分为两个方向。然而，由于尺子的长度比宽度更大，杨氏模量更大，抗弯刚度也更大。因此，在实际计算中我们忽略尺子在 y 方向上的弯曲。于是 $\frac{\partial w}{\partial y} = 0$ ，方程退化为

$$mg(w_0 + h) = D(2 - \sigma) \iint \left(\frac{\partial^2 w}{\partial x^2} \right)^2 dx dy$$



根据理想气体状态方程 $PV = nRT$ ，其中体积、物质的量压强随时间变化， $V = \frac{V_m}{t_m} t$ ，温度为参数。根据气体扩散方程，摩尔通量强度 $J = G \frac{\partial p}{\partial x}$ ， $\frac{dn}{dt} = \iint_S J ds$ ，总控制方程：

$$P \frac{V_m}{t_m} t = RT \iiint G \frac{\partial p}{\partial x} ds dt$$